DAISY: Fault-Guided Assertion Inference for Dafny

Álvaro F. Silva
Faculdade de Engenharia
Universidade do Porto
Porto, Portugal
alvaro.silva@fe.up.pt

Ruben Martins
Computer Science Department
Carnegie Mellon University
Pittsburgh, USA
rubenm@cs.cmu.edu

Alexandra Mendes*
Faculdade de Engenharia
Universidade do Porto
Porto, Portugal
alexandra@archimendes.com

***Abstract*—Formal verification with Dafny provides strong correctness guarantees, but verification failures often require users to determine where helper assertions are needed and to manually write the assertions necessary for verification to succeed. This proof-engineering burden is difficult, time-consuming, and a barrier to the adoption of verification-aware languages. We present DAISY (Dafny Assertion Inference SYstem), a tool for fault-guided assertion inference in Dafny. Given a Dafny program and a target method, DAISY localizes likely assertion sites using verifier diagnostics, LLM-based localization, or their combination. It then annotates the program with assertion placeholders, retrieves relevant in-context examples, generates candidate assertions with an LLM, and validates candidates by running the Dafny verifier. To evaluate DAISY, we extend DafnyBench with verification-failing variants created by removing helper assertions from verified programs. The resulting benchmark distinguishes between programs with one missing assertion, two missing assertions, and all assertions removed, allowing us to evaluate DAISY on increasingly difficult proof failures. On this benchmark, DAISY verifies 86% of programs with a single missing assertion and 36% of programs with two missing assertions. These results show that fault-guided LLM assertion inference can repair many Dafny verification failures. They also show that successful repairs often differ from the original assertions, reducing proof-engineering effort without reconstructing the original proof.**

***Index Terms*—Dafny, Assertion Inference, Fault Localization**

I. Introduction

Verification-aware languages, such as Dafny [11], provide strong correctness guarantees by construction, enabling developers to annotate programs with specifications and automatically prove that the implementation satisfies them. In practice, however, verification rarely succeeds without additional effort: most proofs require numerous helper assertions — assertions that aid the verifier without affecting program behavior. Identifying which assertions are missing and where to place them is challenging. Reports from the industrial verification of cryptographic systems [5] show that *the majority of proof engineering effort is spent addressing failing proofs*, with missing assertions as a primary cause. Likewise, Oliveira et al. [17] identify user challenges with writing annotations. This bottleneck makes assertion inference a crucial step toward improving the scalability and adoption of formal verification.

Large Language Models (LLMs) have recently been applied to reduce this burden by automatically inferring missing annotations [16], [18]. Prior work shows that LLMs can generate assertions, loop invariants, program repairs, and full specifications [13], [21], [24], lowering the manual effort of proof engineering. In particular, Laurel [16] introduced a two-stage approach for Dafny: first localize a likely missing assertion location using verifier diagnostics, and then use retrieval-augmented prompting to generate an assertion. Laurel demonstrated that LLM-assisted assertion inference can unblock verification failures, but it primarily targeted lemmas and single missing assertions. This leaves open the broader problem of supporting verification failures in program code, where multiple helper assertions may be needed.

This paper proposes DAISY (Dafny Assertion Inference SYstem), a tool for helping developers resolve Dafny verification failures with helper assertions. DAISY preserves the original implementation and specification, searching only for auxiliary assertions that help the verifier prove the existing verification goals. By combining fault localization, retrieval-guided LLM generation, and verifier validation, DAISY identifies likely assertion locations and generates helper assertions to verify the Dafny program. DAISY extends prior assertion-inference pipelines along three dimensions. First, it targets Dafny program code in addition to lemmas. Second, it supports inference for one or more assertion locations, enabling repairs that require multiple helper assertions rather than a single missing proof hint. Third, it provides a configurable retrieval component for selecting relevant in-context examples using different retrieval strategies. Together, these features make DAISY a practical framework for studying and deploying assertion inference for Dafny programs.

To evaluate DAISY, we construct a new benchmark dataset of 448 programs by extending DafnyBench [12] with verification-failing variants created by removing helper assertions from verified programs. The benchmark separates cases with one missing assertion, two missing assertions, and all assertions removed, enabling a controlled evaluation across increasingly difficult proof failures. DAISY verifies 86% of single-assertion cases and 36% of two-assertion cases. Many successful repairs differ from the original assertions, demonstrating that multiple valid proof strategies often exist.

Although our implementation targets Dafny, the underlying approach is not Dafny-specific. Many SMT-based verification tools rely on user-provided assertions, verifier diagnostics, and iterative proof debugging [1], [2], [7], [10], [22], [23]. The same combination of fault localization, example retrieval,

* Author order follows a senior-author-last convention.

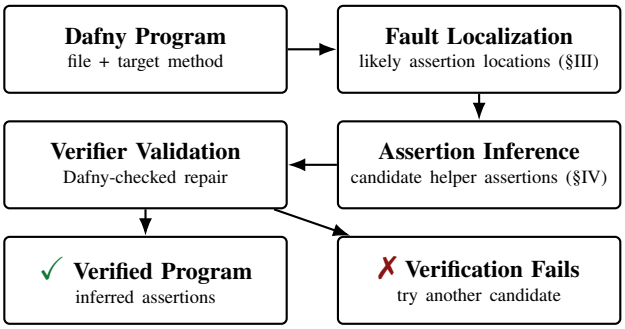


Fig. 1. Overview of DAISY: localizing assertion locations, generating candidates, and validating them with Dafny.

LLM-based assertion generation, and verifier validation can therefore be adapted to other verification-aware languages. Our main contributions are:

- **DAISY (Dafny Assertion Inference SYstem): a fault-guided assertion inference tool.** We present DAISY, a tool that repairs Dafny verification failures caused by missing helper assertions by inferring verifier-checked assertions while preserving the original implementation and specification. The tool and artifacts are available in GitHub with a self-contained artifact in Zenodo.
- **Fault localization for assertion insertion points.** We introduce and evaluate localization strategies that identify one or more likely assertion locations using verifier diagnostics, LLM-based prediction, or their combination.
- **Assertion generation for Dafny program code.** We extend assertion inference beyond lemma-centered settings to Dafny program code, and support configurable retrieval-guided generation with different strategies for selecting relevant in-context examples.
- **Benchmark and empirical evaluation.** We extend DafnyBench with 448 verification-failing variants containing one, two, or all assertions removed, enabling evaluation across proof failures of varying difficulty.

II. Overview

DAISY follows a two-stage pipeline inspired by Laurel [16]. First, *fault localization* identifies likely locations where helper assertions are needed. Second, *assertion inference* generates candidate assertions for those locations and validates them with Dafny. Throughout this process, DAISY preserves the original implementation and specification: the only modifications it attempts are the insertion of auxiliary assertions that help Dafny complete the proof. Figure 1 illustrates the pipeline, whose two main stages are described in detail in §III (Fault Localization) and §IV (Assertion Inference).

We illustrate the pipeline with a concrete example. `FindRange(q, key)` takes a sorted sequence `q` and returns the half-open interval *[left, right)* containing exactly the elements equal to `key`. The example also uses the helper Boolean predicate `Sorted(q)` that returns true if a given sequence is sorted in ascending order.

```
predicate Sorted(q: seq<int>){
  forall i,j :: 0 <=i<=j<|q| ==> q[i]<=q[j]}

method FindRange(q: seq<int>, key: int)
 returns (left: nat, right: nat)
  requires Sorted(q)
  ensures left<=right<=|q|
  ensures forall i :: 0<=i<left ==> q[i]<key
  ensures forall i :: left<=i<right ==> q[i]==key
  ensures forall i :: right<=i<|q| ==> q[i]>key
```

In the method `Main`, a call to `FindRange` correctly computes the range of the value 10. However, Dafny's verifier cannot automatically deduce that the resulting indices are exactly 4 and 7. This failure to prove the assertion `assert i == 4 && j == 7` indicates a need for additional helper assertions.

```
1: method Main(){
2: var q := [1,2,2,5,10,10,10,23];
3: assert Sorted(q);
4: assert 10 in q;
5: var i,j := FindRange(q, 10);
6: assert i == 4 && j == 7 by { /*Fails to prove*/
7:  assert q[0] <= q[1] <= q[2] <= q[3] < 10;
8:}}
```

DAISY proceeds in two stages: localizing likely assertion locations and inferring assertions for them.

Fault Localization. DAISY first identifies candidate locations for helper assertions, using the combined fault localization strategy described in §III. In this example, Dafny reports the diagnostic message *"assertion might not hold"* at line 6, where the final assertion cannot be proved. The verifier-diagnostic strategy uses this message to select the position after line 5, while the LLM-based strategy selects the position after line 6. The combined strategy therefore considers both locations:

```
5 : var i,j := FindRange(q, 10);
after 5: /*<Assertion is Missing Here>*/
6 : assert i == 4 && j == 7 by {
after 6: /*<Assertion is Missing Here>*/
7 : assert q[0] <= q[1] <= q[2] <= q[3] < 10;
```

Assertion Inference. DAISY then prompts an LLM to generate candidate assertions for each placeholder, as described in §IV. For each placeholder, the model returns a ranked list of up to 10 candidate assertions, which are verified in parallel. With sufficient CPU resources, validating ten candidates incurs approximately the same latency as validating one. Consequently, intermediate `pass@k` metrics have little practical significance, as users experience similar latency for one or ten candidates. We therefore report only `pass@10`, which better reflects the tool's practical effectiveness. In this example, the first-ranked candidates for the two placeholders verify the program:

```
5 : var i,j := FindRange(q, 10);
after 5: assert 10 in q[i..j]; /*Added*/
6 : assert i == 4 && j == 7 by {
after 6: assert (j == |q| || q[j] > 10); /*Added*/
7 : assert q[0] <= q[1] <= q[2] <= q[3] < 10;
```

Removing either added assertion causes verification to fail, showing that both assertions are needed. Notably, DAISY's solution differs from the original human-written assertions:

```
var i,j := FindRange(q, 10);
assert i == 4 && j == 7 by {
  assert q[0] <= q[1] <= q[2] <= q[3] < 10;
  assert q[4] == q[5] == q[6] == 10;/*Ground Truth*/
  assert 10 < q[7];/*Ground Truth*/}
```

This example illustrates an important feature of DAISY: successful verification does not require reconstructing the original proof. A solution is valid whenever Dafny proves the original specification after the generated assertions are inserted. Thus, DAISY may produce different, and sometimes fewer, assertions than the human-written proof while preserving the same verification guarantees.

III. Fault Localization

The first stage of our pipeline identifies where helper assertions should be inserted. Given a failing Dafny method and its verifier diagnostic, a localization strategy returns one or more candidate insertion points. We evaluate several strategies for this step: LLM-based localization, verifier-diagnostic localization, and a combined strategy that uses both. The resulting insertion points are converted into assertion placeholders and passed to the assertion inference stage described in §IV.

LLM-based Fault Localization. The LLM-based strategy prompts the model with the line-numbered method and verifier diagnostic, and asks it to return assertion insertion points. The prompt allows multiple positions, since a verification failure may require several helper assertions. We evaluate two LLM-based configurations. The zero-shot configuration, **LLMFL**, uses only the target method and its verifier diagnostic, with no examples. The few-shot configuration, **ExLLMFL**, augments the prompt with three dynamically retrieved examples that illustrate common assertion-placement patterns. These examples are selected using the retrieval strategy described in §IV, which is the best-performing strategy identified in §VI-D.

Verifier-Diagnostic Fault Localization Strategy. This strategy localizes missing helper assertions by mapping Dafny verifier diagnostics to candidate insertion points. It builds on the diagnostic-based rules introduced in Laurel [16], but extends them to cover additional verifier errors. Table I summarizes the diagnostic categories handled by each strategy and the insertion location associated with each category.

DAISY extends the Laurel-derived rules with four additional diagnostic categories. *LoopInvariants* captures failures to prove that a loop invariant holds initially or is preserved by the loop body. *TimeOut* denotes verification conditions that the SMT solver cannot discharge within the allotted time. *SubsetConstraints* covers failures to prove that a value satisfies the constraints of its declared type. *ElementNotInDomain* captures failures to establish membership in a required domain.

We refer to the strategy that uses only the Laurel-derived rules as **DiagFL**, and to the extended strategy that uses all rules in the table as **DiagFL+**. Note that both strategies rely on the verifier's diagnostic messages and can return only *one position* for generating an auxiliary assertion.

TABLE I
ERROR TYPES AND INSERTION STRATEGIES FOR DIAGFL AND DIAGFL+. DIAGFL USES ONLY THE ROWS MARKED WITH *Laurel*. DIAGFL+ EXTENDS DIAGFL AND ALSO USES THE ROWS MARKED DAISY. **B**: INSERT BEFORE THE DIAGNOSTIC LINE l (I.E., AT POSITION $l-1$). **E**: INSERT AT THE END OF THE ENCLOSING BLOCK.

Error type	Origin	Loc.
Assertion, Related, LHSValue, Calc, Constructed	Laurel [16]	B
Postcondition, AssertBy, Forall	Laurel [16]	E
LoopInvariants	DAISY	E
TimeOut, SubsetConstraints, ElementNotInDomain	DAISY	B

Combined Fault Localization. We also evaluate a combined strategy, **BothFL**, that takes the union of the candidate positions returned by **DiagFL+** and **ExLLMFL**. This method considers the union of assertion positions suggested by the heuristic-based strategy and those inferred by the LLM (an example was shown in the Fault Localization paragraph of §II). It combines the reliability of diagnostic heuristics with the contextual flexibility of LLMs, reducing erroneous placements relative to LLM-only localization, while offering greater positional diversity than purely heuristic-based methods.

In summary, for **Fault Localization Strategies** the following variants are explored:

- **DiagFL**: verifier-diagnostic localization using the Laurel-derived rules.
- **DiagFL+**: extended verifier-diagnostic localization using both the Laurel-derived rules and the additional rules introduced in DAISY.
- **LLMFL**: zero-shot LLM-based localization, using only the target method and verifier diagnostic.
- **ExLLMFL**: few-shot LLM-based localization, adding three dynamically retrieved in-context examples.
- **BothFL**: combined localization using the union of positions returned by DiagFL+ and ExLLMFL.

IV. Assertion Inference

After fault localization identifies candidate assertion locations, DAISY passes an annotated version of the failing program to the assertion-inference stage. We call this annotated version the *localized program*: the original failing Dafny program together with explicit placeholders at the insertion points predicted by the fault-localization stage. The role of assertion inference is therefore not to choose where assertions should be inserted, but to generate candidate helper assertions for the locations already selected by fault localization.

For each placeholder, the LLM generates a ranked list of candidate assertions. During validation, DAISY constructs a bounded set of verification attempts from these ranked lists rather than enumerating the full Cartesian product of candidates across placeholders. We then evaluate each candidate individually by inserting it at its corresponding placeholder while leaving all other placeholders empty. We also evaluate aligned combinations by jointly inserting candidates with the same rank across all placeholders. This bounded search

balances exploration of individual and combined repairs with a tractable number of verification attempts.

Example Selection Strategies. All assertion-inference configurations use the same localized program and the same assertion-generation prompt. The only difference between the inference configurations is how the in-context examples are selected. In the zero-shot configuration, no examples are provided. In all few-shot configurations, the prompt includes a few examples selected from a retrieval corpus of previous assertion-repair instances. In our experiments, we use a fixed retrieval budget of $k = 3$ examples, although this is a configurable parameter of the tool. We exclude examples from the same functions, methods, or lemmas, to prevent data leakage. We organize the strategies into three groups: *baselines*, *lexical retrieval*, and *embedding-based retrieval*.

Baselines. The first two strategies measure the effect of examples independently of retrieval quality.

- **NoEx**: No in-context examples are included. This corresponds to a zero-shot prompt baseline.
- **Random**: Examples are selected uniformly at random from the retrieval corpus.

Lexical Retrieval. This strategy tests whether useful examples can be retrieved using explicit token overlap rather than learned semantic representations. This is a natural baseline for Dafny programs because similar proof failures often share concrete syntactic cues, such as assertion keywords, sequence operators, quantified expressions, helper lemma names, or recurring verifier-error phrases.

- **TF–IDF**: Few-shot prompt with the examples with the highest matching score compared with the target using a TF–IDF-based [19] retrieval method. In our implementation, TF–IDF is computed over the Dafny method text itself. Following established practices in software engineering and code retrieval [16], we use TF–IDF as a classical baseline.

Embedding-based Retrieval. The remaining strategies use semantic embeddings of the verifier diagnostic, the program text, or both. For each entry in the retrieval corpus, DAISY precomputes embeddings for the filtered verifier diagnostic and the method body using `jina-embeddings-v2-base-code` [9]. Although other embedding models could be used, we select this model because it is optimized for code-centric tasks and supports an 8,192-token context window, making it suitable for long method bodies and verifier traces. For a new failing program, we embed its error and code, compute cosine similarities with all entries, and combine them as $\alpha \cdot \mathrm{sim}_{\text{error}} + (1-\alpha) \cdot \mathrm{sim}_{\text{code}}$. This approach is inspired by hybrid retrieval systems [4], which integrate multiple semantic embeddings and may also incorporate keyword-based or other retrieval techniques.

- **CodeEmb**: Examples are selected using only code-embedding similarity. This corresponds to the case $\alpha = 0$.
- **MixEmb**$_\alpha$: Examples are selected with $\alpha \in \{0.25, 0.5, 0.75, 1\}$. Larger values give more weight to the verifier diagnostic, while smaller values give more weight to code.

V. Dataset

We built our dataset on top of DafnyBench [12], a benchmark suite of 782 Dafny programs that successfully compile and verify. Starting from this verified corpus, we systematically remove helper assertions to create verification-failing variants, producing instances where our method must infer assertions sufficient to restore verification. Our goal is to evaluate assertion inference primarily in program code rather than only in proof-heavy settings. For this reason, DafnyBench is a natural starting point: unlike assertion-containing datasets such as DafnyGym [16], which are dominated by lemmas and proof-oriented examples, DafnyBench contains a broader collection of Dafny functions, methods, and lemmas. We generated the following:

- **w/o-1** (*without one assertion*): We removed each assertion individually, one at a time. If removal caused verification to fail, we added both the missing assertion and the corresponding program to the **w/o-1** dataset.
- **w/o-2** (*without two assertions*): We removed two assertions simultaneously, considering all pairs containing assertions in the **w/o-1** dataset. Pairs where one or both assertions were not part of **w/o-1** are excluded, as inferring them would be no harder than recovering a single assertion and therefore would not provide meaningful insight for our evaluation.
- **w/o-all** (*without all assertions*): We removed all assertions, retaining only cases with at least three missing assertions to ensure no overlap with **w/o-1** or **w/o-2**. If verification fails, we add the missing assertions and the program to the **w/o-all** dataset.

After applying this procedure, we obtained 236 **w/o-1**, 184 **w/o-2**, and 28 **w/o-all** instances. The **w/o-all** dataset is small, with only 28 cases, which is expected for several reasons. First, many assertions serve primarily as lightweight testing aids, so removing them eliminates the corresponding checks, causing the verification to pass simply because there is nothing left to verify. Second, the number of all-assertion removals is much smaller by construction: a declaration with many assertions can produce many single- or pair-removal instances, but only one all-removal instance. Finally, requiring at least three removed assertions further restricts this category and keeps it distinct from the **w/o-1** and **w/o-2** benchmarks.

Together, these benchmarks let us evaluate assertion inference across increasingly difficult proof failures: a single missing assertion, two interacting missing assertions, and cases where most local proof guidance has been removed.

VI. Evaluation

We designed our evaluation around three research questions:

- **RQ1: How effective is DAISY for assertion inference?** We evaluate fault-localization and assertion-generation strategies on the **w/o-1**, **w/o-2**, and **w/o-all**

benchmarks, showing that combining LLM-based and verifier-diagnostic localization performs best.

- **RQ2: How effective is fault localization?** We compare our fault localization with the ground truth. Ground-truth positions reflect original developer assertions, but are not guaranteed to be optimal or minimal for verification.
- **RQ3: Does example selection improve inference?** We compare strategies for selecting in-context examples, including TF–IDF over program code, code-embedding retrieval, and multi-embedding retrieval over program code and verifier diagnostics.

A. Experimental Setup

We use the 448 Dafny programs (236 **w/o-1**, 184 **w/o-2**, and 28 **w/o-all**) for assertion inference, generated as described in §V. These programs fail to verify unless additional assertions are introduced. To ensure reproducibility, we fixed the toolchain to specific versions of Dafny [11] (fork of v4.10) and Z3 [15] (v4.15.4), with a 60-second verification timeout.

LLM Selection. To select the LLM used in our main experiments, we performed a pre-test on the assertion inference phase. We used a random subset of 30 benchmark instances and supplied each model with **ExactFL** positions, i.e., the ground-truth location which corresponds to the original locations of the removed assertions in the verified programs. Evaluating all candidate models on the full dataset was computationally prohibitive, so we used this pre-test as a cost-benefit analysis. We compared several models (Claude Haiku/Sonnet/Opus 4.5, GPT 5.2, GPT 5 Mini, GPT 4.1, DeepSeek R1, Qwen3 Coder 480B/30B and Llama 3.3) through the Amazon Bedrock and OpenAI APIs in January 2026.

Considering both performance and cost, some models are dominated by others (lower accuracy at higher cost). After filtering, the most relevant candidates are Claude Opus 4.5, GPT-5.2, Claude Haiku 4.5, and Qwen3 Coder 480B. Although OPUS has the highest accuracy (73%), its cost is substantially higher. For the core ablation studies, we focus on HAIKU, which offers strong performance (67%) at a lower cost (6.54 cents per run). HAIKU slightly outperformed Sonnet on this small dataset, suggesting it is capable of detailed experiments, though this difference could be due to chance. The full ablation study with HAIKU is estimated at around $50, while running the same with OPUS would scale to $250, making it impractical. For the best methods, we also compare with OPUS to assess differences with a higher-capacity model.

Each evaluated configuration combines a fault-localization strategy with an example-retrieval mechanism. Cost estimates are shown for Claude Haiku 4.5 ($1 per 1M input tokens, $5 per 1M output tokens), with default generation parameters. Table II reports the estimated cost for evaluating one configuration on the full dataset. We compare our methods to a baseline, **FullResponse**, which queries the LLM for the complete output in a single step. This one-shot approach is not only less reliable, sometimes modifying unrelated code, but also more expensive, as larger outputs require more tokens. To allow a fairer comparison, **FullResponse(10)** requests 10 outputs per prompt, matching the number of alternatives generated by our methods. With the adjustments, FullResponse remains significantly more costly.

TABLE II
COST ESTIMATES FOR ALL EVALUATED CONFIGURATIONS USING CLAUDE HAIKU 4.5. INPUT/OUTPUT TOKENS ARE IN MILLIONS (M) AND THOUSANDS (K), RESPECTIVELY, CONSIDERING CLAUDE HAIKU 4.5 BASE PRICE

Model	# Prompts	Input (M)	Output (K)	Cost ($)
DiagFL+	448	1.0	108	1.6
LLMFL	896	1.6	109	2.1
ExLLMFL	896	2.2	109	2.8
BothFL	896	2.2	109	2.8
FullResponse (1)	448	1.1	185	2.0
FullResponse (10)	4480	11.0	1850	20

This two-stage separation, inspired by Laurel [16], preserves verification soundness while enabling explicit evaluation of each stage, and lower costs than FullResponse prompting.

Fault Localization Evaluation. To evaluate the precision of the localization strategies, we categorize each predicted insertion point based on the verifier's response when the ground-truth assertion is inserted at that specific location. We define three mutually exclusive categories:

- **Invalid**: where insertion causes a syntax error;
- **Partial**: where insertion is syntactically valid but verification still fails;
- **Valid**: where insertion is syntactically valid and verification succeeds when using the ground-truth assertion.

This classification moves beyond simple edit-distance metrics. In Dafny an assertion may be valid at several locations as long as the required facts are in scope and the assertion appears before it is needed. We observed in our experiments that many assertions have multiple valid locations, sometimes up to 25, and that valid locations for the same assertion can be separated by more than 40 lines.

We compute the set of **Valid** positions using a perturbation procedure, where we move an assertion across all possible insertion points, and record the positions that restore verification. Because this procedure requires many verifier calls, we apply it only to **w/o-1** and **w/o-2**. For programs with two missing assertions, we fix one ground-truth assertion at its original location and move the other. We then repeat the process with the two assertions swapped. The union of all successful positions defines the valid-location set for that program.

While this metric provides a robust evaluation framework, it is not exhaustive. The LLM may generate a different assertion, or a combination of assertions, that successfully verifies even if placed at a location where the original ground-truth assertion would fail. We analyze this in §VI-C.

Statistical Testing. To assess whether performance differences between methods are statistically significant, we apply McNemar's test with a significance level of $\alpha = 0.05$. We use this test to compare selected pairs of approaches, primarily contrasting LLM-based with heuristic-based methods, as well as comparing the best-performing method against alternatives.

TABLE III
VERIFICATION SUCCESS RATE ACROSS BENCHMARK CATEGORIES USING THE BEST RETRIEVAL STRATEGY, MIXEMB$_{0.25}$.

Model	Localization	Benchmarks			
		w/o-1	w/o-2	w/o-all	Combined
HAIKU	DiagFL	52.5%	6.5%	10.7%	31.0%
	DiagFL+	57.6%	3.3%	10.7%	32.4%
	LLMFL	49.2%	9.8%	**21.4%**	31.2%
	ExLLMFL	53.4%	9.8%	17.9%	33.3%
	BothFL	**67.4%**	**14.1%**	**21.4%**	**42.6%**
	ExactFL	71.6%	26.1%	25.0%	50.0%
OPUS	DiagFL+	75.4%	6.0%	21.4%	43.5%
	ExLLMFL	78.0%	33.2%	**28.6%**	56.5%
	BothFL	**86.0%**	**35.9%**	21.4%	**61.4%**
	ExactFL	91.1%	65.2%	50.0%	77.9%

B. RQ1: How effective is DAISY for assertion inference?

To evaluate end-to-end assertion inference, we combine each fault-localization strategy with the best example-retrieval strategy, **MixEmb**$_{0.25}$ (identified in §VI-D). We measure whether the generated assertions restore verification on the **w/o-1**, **w/o-2**, and **w/o-all** benchmarks. As a reference point, we also include **ExactFL**, which fixes the insertion points to the original assertion locations, isolating the effects of assertion generation and validation from localization.

We first evaluate all fault-localization strategies using HAIKU. To control cost, we evaluate OPUS only on the best methods: the best verifier-diagnostic strategy, **DiagFL+**, the best LLM-based strategy, **ExLLMFL**, and their combination, **BothFL**. Table III summarizes the verification success rates. The ordering of results when using Both HAIKU and OPUS produce the same ranking. **ExLLMFL** outperforms **DiagFL+**, and the best is **BothFL**.

Analyzing HAIKU results in detail, we observe that **DiagFL+** improves upon **DiagFL**, and that **ExLLMFL** outperforms **LLMFL**. Overall, the performance of **ExLLMFL** and **DiagFL+** is statistically indistinguishable. However, **ExLLMFL** shows a notable improvement on the **w/o-2** dataset, increasing the success rate from 3% to 10%. The **BothFL** method consistently achieves the best overall performance among the evaluated localization approaches. Compared to **DiagFL+**, the improvement is statistically significant, as is the comparison with **ExLLMFL**.

With OPUS, the ranking of methods remains consistent, with **BothFL** performing best across all benchmarks. Its improvements over **DiagFL+** and **ExLLMFL** are statistically significant. Moreover, **ExLLMFL** significantly outperforms **DiagFL+**, with the effect being particularly pronounced on the **w/o-2** dataset, where performance rises from 6% to 33%.

Multiple Missing Assertions. Performance decreases as more assertions are removed. For example, OPUS **BothFL** verifies 86% of **w/o-1** programs but only 36% of **w/o-2** programs. This drop is larger than expected under an independence assumption: if the two assertions were inferred independently, the **w/o-2** success rate would be roughly 74% (0.86^2). The observed 36% suggests that missing assertions are often interdependent, making multi-assertion inference substantially harder than solving two independent single-assertion tasks. Another contributing factor is our bounded pairing strategy. Rather than evaluating the full Cartesian product of candidate assertions, we consider only a limited set of pairings for computational efficiency. Consequently, a correct combination of assertions may exist but remain untested.

The **w/o-all** results are harder to interpret because this benchmark contains only 28 instances. In some cases, success does not decrease monotonically relative to **w/o-2**, likely because the small sample is sensitive to the particular difficulty of the selected programs. We therefore treat **w/o-all** as a stress test rather than as a primary basis for comparing methods. Still, the fact that several methods verify some **w/o-all** programs shows that restoring verification does not require reproducing the original proof structure exactly; alternative assertions and insertion points can also be sufficient.

Effect of Combined Localization. The strong performance of **BothFL** comes from combining complementary localization signals. Verifier-diagnostic localization is precise but limited to fixed error-message patterns, while LLM-based localization can exploit broader program context but may propose less reliable positions. Taking the union of both sets of positions creates a small portfolio of candidate locations that is more robust than either source alone.

C. RQ2: How effective is fault localization?

Localization Quality. We evaluate fault localization using the validity-based classification from §VI-A. Figure 2 summarizes the localization results. The **DiagFL** strategy leaves approximately 25% of cases unsupported, mainly because it does not handle some verifier errors, such as loop-invariant failures. Our extended strategy, **DiagFL+**, reduces unsupported cases to 7% and achieves the highest valid-position rate among individual methods, with 72% valid placements. The LLM-based **ExLLMFL** method also performs well, reaching 61% valid placements. Overall, the combined **BothFL** strategy performs best, identifying valid placements in 80% of cases.

Combined Localization. The strong performance of **BothFL** comes from combining complementary signals. Verifier-diagnostic localization is precise when the error message matches a supported pattern, but it can be too local or fail to return a position. LLM-based localization can use broader program context, but its predictions are less deterministic. Taking the union of both predictions gives a small portfolio of candidate locations that is more robust than either source alone. The following example illustrates this complementarity:

```
method testRawsort() {
  var a : array<T> := new T[] [3, 5, 1];
  /*<Assertion is Missing Here>*/ ExLLMFL
  rawsort(a);
  /*<Assertion is Missing Here>*/ DiagFL+
  assert a[..] == [1, 3, 5];}
```

Here, **DiagFL+** places the missing assertion immediately before the failing check, following the verifier diagnostic.

TABLE IV
VERIFICATION SUCCESS RATE FOR EXAMPLE-SELECTION STRATEGIES USING EXACTFL FAULT LOCALIZATION AND THE HAIKU MODEL.

Example Selection	w/o-1	w/o-2	w/o-all	Combined
NoEx	59.3%	20.7%	14.3%	40.6%
Random	59.7%	20.7%	17.9%	41.1%
TF–IDF	70.3%	23.9%	**28.6%**	48.7%
CodeEmb	69.9%	23.9%	21.4%	48.0%
$\text{MixEmb}_{0.25}$	71.6%	26.1%	25.0%	**50.0%**
$\text{MixEmb}_{0.50}$	68.2%	**28.3%**	25.0%	49.1%
$\text{MixEmb}_{0.75}$	**72.9%**	24.5%	21.4%	49.8%
$\text{MixEmb}_{1.00}$	68.2%	25.0%	17.9%	47.3%

However, the useful assertion must be introduced before the call to `rawsort(a)`, where it can relate the initial array contents to the subsequent sorting call. **ExLLMFL** identifies this earlier location. By combining both predictions, **BothFL** avoids committing to a single localization strategy and reduces the risk of missing useful insertion points.

Impact on Verification. Localization quality also affects end-to-end verification. We observed in our experiments that positions classified as *valid* are 60% more likely than *partial* positions to lead to successful verification. This supports our validity-based metric as a more meaningful indicator of localization quality than distance to the ground truth.

D. RQ3: Does example selection improve inference?

To isolate the impact of example extraction from fault localization, we evaluated several retrieval methods using the **ExactFL** positions as a fixed baseline. In addition to standard baselines, we evaluate a hybrid retrieval strategy that combines code and error-message embeddings via a weighted cosine similarity metric, parameterized by α (**ExactFL-MixEmb**$_\alpha$).

Table IV summarizes the results. **NoEx** and **Random** are statistically indistinguishable. This suggests that arbitrary examples provide little benefit. In contrast, all retrieval-based strategies improve over these baselines, showing that relevant examples are useful for assertion inference. The best-performing method is **MixEmb**$_{0.25}$, which achieves a 50% success rate. This is a statistically significant improvement over **NoEx**, which only had a 41% success rate. However, differences among retrieval-based methods are small. For example, **MixEmb**$_{0.25}$ does not significantly outperform the strongest non-MixEmb baseline, **TF–IDF**. This suggests that retrieving relevant context matters more than the particular retrieval method. We therefore used **MixEmb**$_{0.25}$ as the default example-selection strategy in the evaluation. It provides examples both for assertion inference and for the few-shot **ExLLMFL** localization method.

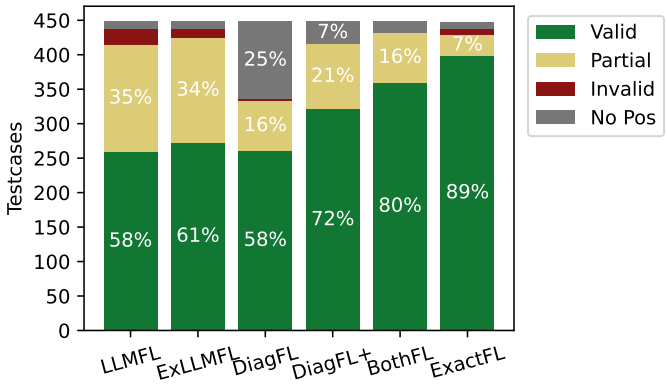


Fig. 2. Assertion insertion-position classification across localization strategies on the combined dataset of 448 Dafny programs using the HAIKU model.

VII. DISCUSSION AND LIMITATIONS

Our best approach **BothFL** is unable to solve 14% of the cases for the **w/o-1** dataset, and even if we had perfect fault localization, we would still have 9% unsolved cases for this dataset, indicating that some assertions are inherently challenging to infer. We manually analyzed all **w/o-1** assertions that none of the methods could solve and identified categories that suggest directions for improvement.

Failure Modes. Several failures require proof structure beyond a single straightforward assertion. In some examples, the LLM correctly identified the required assertion but failed to provide a `by`-justification. For instance, in: `assert c <= x by { reveal A; }` the model inferred `assert c <= x;`, but did not recognize the need to reveal the named clause `A` explicitly. A stronger integration of named assumptions and lemma references could likely solve such errors.

Many failures involved reasoning about sequence slicing or concatenation, which is common in Dafny proofs. Typical missing assertions include: `assert rest == rest[0..1] + rest[1..];`, `assert r == r[..k] + r[k..];`. Instead of producing these direct equalities, the model often attempted overcomplicated multiset reasoning or type-incorrect variations. Improvements could come from incorporating explicit pattern templates for sequence equalities or augmenting the prompt with more examples of this type.

Context Limitations. DAISY prompts the model primarily with the target declaration, so relevant auxiliary functions, lemmas, types, or named assumptions may be omitted. Some assertions therefore depend on information unavailable to the model. Including surrounding definitions through targeted context retrieval could improve performance, but would increase prompt size, cost, and the risk of adding irrelevant context.

Near Misses and Repair Opportunities. Some failures were close to correct. For example, the model sometimes inferred the right algebraic structure but made a small arithmetic error, such as an off-by-one constant.

Evaluation Limitations. As with other LLM-based evaluations, our results may be affected by model nondeterminism and potential data leakage from pretraining. Since each configuration was run once, the reported results may contain some sampling noise. However, repeated runs are often prohibitively expensive in this setting [12], [16]. To reduce dependence on any single LLM output, each prompt requests up to 10 candidate assertions per placeholder. We therefore

report verification success under a fixed 10-candidate budget: a benchmark instance is counted as solved if any verifier-validation attempt constructed from these candidates succeeds.

Data leakage is another threat: models may have seen similar Dafny code or verification examples during pretraining, potentially inflating performance through memorization. We partially mitigate this concern with two observations. First, performance improves with prompt enrichment, such as adding retrieved examples, suggesting that model outputs depend on the provided context. Second, inferred assertions and insertion points often differ from the original ground truth, indicating that the model frequently synthesizes new verification strategies rather than simply reproducing memorized assertions.

VIII. Related Work

Complete Program Synthesis. Recent work integrates LLMs into Dafny verification workflows. Clover [21] generates code and specifications from natural language and checks their consistency with documentation and annotations. Other systems co-evolve programs, specifications, and tests [13], synthesize verified Dafny methods under different prompting strategies [14], or use verifier-guided search for multi-step Dafny programs [3]. Unlike these approaches, DAISY preserves the existing Dafny code and focuses on localizing and inferring helper assertions.

Invariant Inference. Invariant generation is a key subproblem in program synthesis for deductive verification, as invariants are essential for reasoning about loops [8]. Compared to full program synthesis, it is simpler and more tractable. Research in this area has achieved significant progress. For Dafny, Faria et al. [6] developed an LLM-based invariant inference tool that verifies 92% of cases on the first attempt and 95% within five attempts.

Assertion Inference. The closest work to DAISY is Laurel [16], which combines verifier-guided localization with LLM-based assertion generation for Dafny. Laurel identifies potential fix sites by analyzing Dafny's verifier error messages, applies heuristics to select promising locations, and retrieves similar examples to guide LLM prompting. Our work differs from Laurel in several ways: (1) it evaluates a pipeline for multi-assertion inference and explores methods that directly generate multiple assertion candidates; (2) it uses LLMs not only for generation but also for predicting assertion insertion positions; (3) it shows that combining LLM-predicted locations with verifier-diagnostic localization yields better results; and (4) it introduces a framework for evaluating the quality of position inference independently of assertion generation.

Proof Repair. Silva et al. [20] studied GPT-4 for lemma discovery and proof sketching in Dafny, showing that even imperfect suggestions can help complete proofs. Poesia et al. [18] inferred both loop invariants and helper assertions with a fine-tuned model, but rely on exhaustive testing across valid insertion points, which is costly for developer tools. Loughridge et al. [12] use LLMs to infill missing DafnyBench annotations, including loop invariants and assertions, reporting a 68% success rate.

We also tried prompt-based infilling, but found it unreliable in our setting. Instead, DAISY first localizes likely assertion positions and then generates multiple candidate assertions in a single query. The difference in performance relative to DafnyBench-based infilling is partly due to dataset construction. Loughridge et al. evaluate on all removed annotations, whereas we keep only cases where removing an assertion causes a genuine verification failure. This excludes easier cases whose removal has no effect on verification, yielding a more challenging benchmark.

IX. Conclusion

We presented DAISY, an LLM-assisted tool for inferring helper assertions in Dafny. DAISY separates assertion localization from assertion generation, enabling systematic evaluation of verifier-diagnostic localization, LLM-based localization, retrieval strategies, and model capabilities. We also introduced a curated DafnyBench-derived benchmark with variants where one, two, or all helper assertions are removed.

Our results show that combining LLM-based localization with verifier diagnostics yields the best overall performance. With combined localization and multi-embedding retrieval, DAISY verifies 86% of programs with one missing assertion, 36% with two missing, and 61% overall. The results also show that successful verification does not require reconstructing the original proof: inferred assertions and insertion points can differ from the human-written ones while still verifying.

Challenges remain for by-proofs, algebraic reasoning, multi-line assertions, and proofs requiring auxiliary definitions or lemmas. Still, DAISY shows that fault-guided LLM assertion inference can repair a substantial fraction of Dafny verification failures, and the same localization-generation-validation structure may extend to other SMT-based verification frameworks.

Acknowledgments

Álvaro Silva and Alexandra Mendes were partially funded by National Funds through the FCT - Fundação para a Ciência e a Tecnologia, I.P. (Portuguese Foundation for Science and Technology) within the project VeriFixer, with reference 2023.15557.PEX (DOI: 10.54499/2023.15557.PEX). Álvaro Silva was also partially supported by Fundação para a Ciência e a Tecnologia (Portuguese Foundation for Science and Technology) through the Carnegie Mellon Portugal Program under the fellowship reference PRT/BD/155045/2024.

Ruben Martins was partially supported by the National Science Foundation (NSF) under Award CCF2427581 and DARPA Agreement FA8750-24-9-1000.

Alexandra Mendes was also partially supported by an Amazon Research Award, Fall 2024.

References

[1] Ahrendt, W., Beckert, B., Bubel, R., Hähnle, R., Schmitt, P.H., Ulbrich, M. (eds.): Deductive Software Verification - The KeY Book - From Theory to Practice, Lecture Notes in Computer Science, vol. 10001. Springer (2016). https://doi.org/10.1007/978-3-319-49812-6, https://doi.org/10.1007/978-3-319-49812-6

[2] Astrauskas, V., Müller, P., Poli, F., Summers, A.J.: Leveraging rust types for modular specification and verification. Proc. ACM Program. Lang. **3**(OOPSLA), 147:1–147:30 (2019). https://doi.org/10.1145/3360573, https://doi.org/10.1145/3360573

[3] Brandfonbrener, D., Henniger, S., Raja, S., Prasad, T., Loughridge, C.R., Cassano, F., Hu, S.R., Yang, J., Byrd, W.E., Zinkov, R., Amin, N.: VerMCTS: Synthesizing Multi-Step Programs using a Verifier, a Large Language Model, and Tree Search. In: Workshop on Mathematical Reasoning and AI at NeurIPS (2024)

[4] Chen, S., Zhao, Z., Chen, J.: Each to their own: Exploring the optimal embedding in RAG. CoRR **abs/2507.17442** (2025). https://doi.org/10.48550/ARXIV.2507.17442, https://doi.org/10.48550/arXiv.2507.17442

[5] Dodds, M.: Formally verifying industry cryptography. IEEE Secur. Priv. **20**(3), 65–70 (2022). https://doi.org/10.1109/MSEC.2022.3153035, https://doi.org/10.1109/MSEC.2022.3153035

[6] Faria, J.P., Trigo, E., Abreu, R.: Automatic generation of loop invariants in dafny with large language models. In: Hojjat, H., Caltais, G. (eds.) Fundamentals of Software Engineering - 11th IFIP WG 2.2 International Conference, FSEN 2025, Västerås, Sweden, April 7-8, 2025, Proceedings. Lecture Notes in Computer Science, vol. 15593, pp. 138–154. Springer (2025). https://doi.org/10.1007/978-3-031-87054-5_10, https://doi.org/10.1007/978-3-031-87054-5_10

[7] Filliâtre, J., Paskevich, A.: Why3 - where programs meet provers. In: Felleisen, M., Gardner, P. (eds.) Programming Languages and Systems - 22nd European Symposium on Programming, ESOP 2013, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2013, Rome, Italy, March 16-24, 2013. Proceedings. Lecture Notes in Computer Science, vol. 7792, pp. 125–128. Springer (2013). https://doi.org/10.1007/978-3-642-37036-6_8, https://doi.org/10.1007/978-3-642-37036-6_8

[8] Floyd, R.W.: Assigning meanings to programs. In: Colburn, T.R., Fetzer, J.H., Rankin, T.L. (eds.) Program Verification - Fundamental Issues in Computer Science, Studies in Cognitive Systems, vol. 14, pp. 65–81. Springer Netherlands (1993). https://doi.org/10.1007/978-94-011-1793-7_4, https://doi.org/10.1007/978-94-011-1793-7_4

[9] Günther, M., Ong, J., Mohr, I., Abdessalem, A., Abel, T., Akram, M.K., Guzman, S., Mastrapas, G., Sturua, S., Wang, B., Werk, M., Wang, N., Xiao, H.: Jina embeddings 2: 8192-token general-purpose text embeddings for long documents. CoRR **abs/2310.19923** (2023). https://doi.org/10.48550/ARXIV.2310.19923, https://doi.org/10.48550/arXiv.2310.19923

[10] Lattuada, A., Hance, T., Cho, C., Brun, M., Subasinghe, I., Zhou, Y., Howell, J., Parno, B., Hawblitzel, C.: Verus: Verifying rust programs using linear ghost types. Proc. ACM Program. Lang. **7**(OOPSLA1), 286–315 (2023). https://doi.org/10.1145/3586037, https://doi.org/10.1145/3586037

[11] Leino, K.R.M.: Dafny: An automatic program verifier for functional correctness. In: Clarke, E.M., Voronkov, A. (eds.) Logic for Programming, Artificial Intelligence, and Reasoning - 16th International Conference, LPAR-16, Dakar, Senegal, April 25-May 1, 2010, Revised Selected Papers. Lecture Notes in Computer Science, vol. 6355, pp. 348–370. Springer (2010). https://doi.org/10.1007/978-3-642-17511-4_20, https://doi.org/10.1007/978-3-642-17511-4_20

[12] Loughridge, C., Sun, Q., Ahrenbach, S., Cassano, F., Sun, C., Sheng, Y., Mudide, A., Misu, M.R.H., Amin, N., Tegmark, M.: Dafnybench: A benchmark for formal software verification. Trans. Mach. Learn. Res. **2025** (2025), https://openreview.net/forum?id=yBgTVWccIx

[13] Mirchev, M., Costea, A., Singh, A.K., Roychoudhury, A.: Assured automatic programming via large language models. CoRR **abs/2410.18494** (2024). https://doi.org/10.48550/ARXIV.2410.18494, https://doi.org/10.48550/arXiv.2410.18494

[14] Misu, M.R.H., Lopes, C.V., Ma, I., Noble, J.: Towards ai-assisted synthesis of verified dafny methods. Proc. ACM Softw. Eng. **1**(FSE), 812–835 (2024). https://doi.org/10.1145/3643763, https://doi.org/10.1145/3643763

[15] de Moura, L.M., Bjørner, N.S.: Z3: an efficient SMT solver. In: Ramakrishnan, C.R., Rehof, J. (eds.) Tools and Algorithms for the Construction and Analysis of Systems, 14th International Conference, TACAS 2008, Held as Part of the Joint European Conferences on Theory and Practice of Software, ETAPS 2008, Budapest, Hungary, March 29-April 6, 2008. Proceedings. Lecture Notes in Computer Science, vol. 4963, pp. 337–340. Springer (2008). https://doi.org/10.1007/978-3-540-78800-3_24, https://doi.org/10.1007/978-3-540-78800-3_24

[16] Mugnier, E., Gonzalez, E.A., Polikarpova, N., Jhala, R., Zhou, Y.: Laurel: Unblocking automated verification with large language models. Proc. ACM Program. Lang. **9**(OOPSLA1), 1519–1545 (2025). https://doi.org/10.1145/3720499, https://doi.org/10.1145/3720499

[17] Oliveira, F., Mendes, A., Carreira, C.: What challenges do developers face when using verification-aware programming languages? In: Brito, A., de Oliveira Moraes, R.L., Sui, Y., Trubiani, C., Aires, F., Ivaki, N. (eds.) 36th IEEE International Symposium on Software Reliability Engineering, ISSRE 2025, São Paulo, Brazil, October 21-24, 2025. pp. 203–214. IEEE (2025). https://doi.org/10.1109/ISSRE66568.2025.00031, https://doi.org/10.1109/ISSRE66568.2025.00031

[18] Poesia, G., Loughridge, C., Amin, N.: dafny-annotator: Ai-assisted verification of dafny programs. CoRR **abs/2411.15143** (2024). https://doi.org/10.48550/ARXIV.2411.15143, https://doi.org/10.48550/arXiv.2411.15143

[19] Ramos, J., et al.: Using TF-IDF to determine word relevance in document queries. In: Proceedings of the first instructional conference on machine learning. vol. 242, pp. 29–48 (2003)

[20] Silva, Á.F., Mendes, A., Ferreira, J.F.: Leveraging large language models to boost dafny's developers productivity. In: Plat, N., Gnesi, S., Furia, C.A., Lopes, A. (eds.) Proceedings of the 2024 IEEE/ACM 12th International Conference on Formal Methods in Software Engineering (FormaliSE), Lisbon, Portugal, April 14-15, 2024. pp. 138–142. ACM (2024). https://doi.org/10.1145/3644033.3644374, https://doi.org/10.1145/3644033.3644374

[21] Sun, C., Sheng, Y., Padon, O., Barrett, C.W.: Clover: Closed-loop verifiable code generation. In: Avni, G., Giacobbe, M., Johnson, T.T., Katz, G., Lukina, A., Narodytska, N., Schilling, C. (eds.) AI Verification - First International Symposium, SAIV 2024, Montreal, QC, Canada, July 22-23, 2024, Proceedings. Lecture Notes in Computer Science, vol. 14846, pp. 134–155. Springer (2024). https://doi.org/10.1007/978-3-031-65112-0_7, https://doi.org/10.1007/978-3-031-65112-0_7

[22] Swamy, N., Hritcu, C., Keller, C., Rastogi, A., Delignat-Lavaud, A., Forest, S., Bhargavan, K., Fournet, C., Strub, P., Kohlweiss, M., Zinzindohoue, J.K., Zanella-Béguelin, S.: Dependent types and multi-monadic effects in F. In: Bodík, R., Majumdar, R. (eds.) Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2016, St. Petersburg, FL, USA, January 20 - 22, 2016. pp. 256–270. ACM (2016). https://doi.org/10.1145/2837614.2837655, https://doi.org/10.1145/2837614.2837655

[23] Vazou, N., Seidel, E.L., Jhala, R.: Liquidhaskell: experience with refinement types in the real world. In: Swierstra, W. (ed.) Proceedings of the 2014 ACM SIGPLAN symposium on Haskell, Gothenburg, Sweden, September 4-5, 2014. pp. 39–51. ACM (2014). https://doi.org/10.1145/2633357.2633366, https://doi.org/10.1145/2633357.2633366

[24] Wu, V., Mendes, A., Abreu, A.: Specification-guided repair of arithmetic errors in dafny programs using llms. In: Bianculli, D., Gómez-Martínez, E. (eds.) Software Engineering and Formal Methods - 23rd International Conference, SEFM 2025, Toledo, Spain, November 10-14, 2025, Proceedings. Lecture Notes in Computer Science, vol. 16192, pp. 261–278. Springer (2025). https://doi.org/10.1007/978-3-032-10444-1_16, https://doi.org/10.1007/978-3-032-10444-1_16